

Key Setup 단계

x : Committee의 **master secret**

x_i : Committee member i 가 가진 **secret share**

- K_{audit} 의 직접적인 share가 아닙니다.
- 복호화에 필요한 shared point Z 를 복원할 때 사용합니다.

G : Jubjub의 고정 **generator point**

PK : 모든 AuditRecord가 공통으로 사용하는 **Committee public key**

- $PK = xG$
- Participant는 PK 만으로 감사 정보를 암호화할 수 있습니다.

공급망 Event가 발생할 때 Participant가 생성하는 값

r : Participant가 AuditRecord마다 새로 생성하는 **암호화 난수**

R_1 : AuditRecord에 저장하는 **public point**

- $R_1 = rG$
- AuditRecord마다 새로운 r 을 사용하므로 R_1 도 달라집니다.

Z : Participant가 암호화할 때 계산하는 **private shared point**

- $Z = rPK$
- Z 는 public input이나 AuditRecord에 저장하지 않습니다.

K_{audit} : AuditRecord 하나의 평문을 가리는 **private Field masking key**

- $K_{\text{audit}} = H(Z)$
- AuditRecord마다 서로 다른 Z 에서 파생되므로 서로 다른 K_{audit} 을 사용합니다.

k_j : AuditRecord 안의 j 번째 평문에 사용하는 **위치별 마스크**

- $k_j = H(K_{\text{audit}}, j)$

M_j : 암호화할 j 번째 **감사 평문**

C_j : AuditRecord에 저장하는 j 번째 암호문

- $C_j = M_j + k_j \pmod{p}$

AuditRecord에 실제로 저장하는 암호화 정보:

- R_1
- $C_0, C_1, \dots, C_{n-1}$

AuditRecord에 저장하지 않는 값:

- r
- Z
- K_{audit}
- k_j
- 평문 M_j

복호화를 진행하는 경우

Auditor는 복호화하려는 AuditRecord에서 다음 값을 읽습니다.

- public point R_1
- 암호문 $C_0, C_1, \dots, C_{n-1}$

Committee member i 는 자기 secret share x_i 와 R_1 으로 **partial decryption point**를 계산합니다.

D_i : Committee member i 가 계산하는 partial decryption point

- $D_i = x_i R_1$
- shared point Z 를 복원하기 위한 타원곡선 점입니다.

Participant가 암호화할 때 계산한 Z 와 Auditor가 복호화할 때 복원한 Z 는 같습니다.

Auditor가 평문을 복원하는 과정

Auditor는 복원한 Z 에서 같은 K_{audit} 을 계산합니다.

- $K_{\text{audit}} = H(Z)$

각 위치의 마스크를 다시 계산합니다.

- $k_j = H(K_{\text{audit}}, j)$

암호문에서 마스크를 빼서 평문을 복원합니다.

- $M_j = C_j - k_j \pmod{p}$

AuditRecord 사이에서 같은 값과 다른 값

모든 AuditRecord가 공통으로 사용하는 값:

- Committee master secret에서 파생된 PK
- Jubjub generator G

AuditRecord마다 새로 생성되는 값:

- r
- R_1
- Z
- K_{audit}
- 위치별 k_j
- 암호문 C_j

따라서 현재 M9은 다음 구조입니다.

- Committee public key PK : 모든 AuditRecord에서 동일
- Field masking key K_{audit} : AuditRecord마다 다름
- 위치별 마스크 k_j : 같은 AuditRecord 안에서도 위치마다 다름

Event Circuit이 검증하는 내용

proof가 성공하면 다음 관계가 성립합니다.

- AuditRecord의 암호문에는 실제 부모와 실제 output 소비 식별값이 들어 있습니다.

Circuit은 Participant가 임의의 값을 암호화하지 못하도록 다음을 확인합니다.

1. 실제 private input에서 부모 cm 또는 rv 를 계산합니다.

2. 실제 output과 owner secret에서 미래 nf 또는 $rvnf$ 를 계산합니다.
3. private witness r 로 $R_1 = rG$ 를 계산합니다.
4. 고정된 Committee public key로 $Z = rPK$ 를 계산합니다.
5. Z 에서 K_{audit} 을 계산합니다.
6. 위치별 마스크 k_j 를 계산합니다.
7. 실제 감사 평문으로 계산한 C_j 가 public input의 암호문과 같은지 확인합니다.